Chapter 3

Sequential Data: Linked Lists

We have encountered the concept of *arrays* a few times by now: the `main` method takes an array of strings and there is a `System.out.print(...)` variant that takes an array of characters. However, arrays have one major weakness that makes them a poor choice for many applications: they are fixed in size. You cannot add elements to nor remove elements from an array.

In this section, we discuss the simplest kind of *recursive data structure*, the *linked list*, which represents a sequence of data as a chain of nodes, where a node is a container holding one piece of data and a reference to the next node in the chain. Nodes can be inserted into or removed from the chain in order to add elements into or remove elements from the sequence. The core of this structure is formed from a recursive `Node` class. A reference to the beginning of the list is held by a wrapping `LinkedList` class. This wrapper class exists for two reasons: first, it promotes encapsulation by hiding the nodes as an implementation detail, and second, it allows for update methods to prepend to the beginning of the list. Without the wrapper, if you were directly holding a reference to the beginning of the list, then prepending data would require reassigning that reference to point to the new beginning. It also makes it much easier to represent an empty list, which has no beginning.

This chapter assumes a structure where the `LinkedList` class contains the `Node` class, as a nested, `private static` member. That is, the `Node` class is declared as a `private static class` inside of `LinkedList`. As long as no nodes are ever directly returned, this hides away the implementation detail that nodes are even used at all. Java's access control system is designed such that in this case, a `LinkedList` has full access to all members of a `Node`, even its private fields and methods. So any methods defined on `Node` can be marked as private just like any other helper methods that should not be exposed to the outside world.

There are any ways to represent the list structure. Here, we will consider and compare two approaches: one based on *persistence* where the internal nodes are immutable, sometimes called the *functional style*, and one based on *ephemerality*

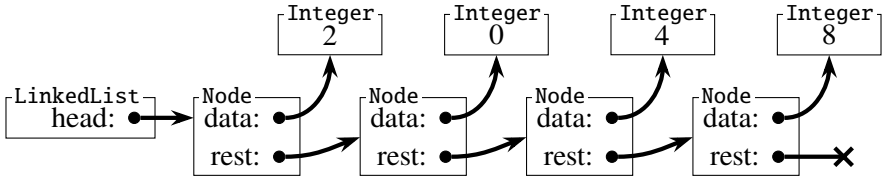

Figure 3.1: A linked list.

where the internal nodes can be modified. Figure 3.1 represents a list containing the integers 2, 0, 4, and 8, in that order. The cross-headed arrow out of the final node represents a null reference, indicating that no further nodes exist in the chain.

3.1 The Persistent Approach

The functional style is so named because, like a mathematical function, it works by manipulating immutable data and returning new values as a result. This matches the strategy used earlier when squaring numbers. The wrapping `LinkedList` will have a mutable `head` that points to whatever the current beginning of the list is, but internally each `Node` will be entirely immutable. Changes are made by creating new structures that share some state with the original structure, but may differ in some components.

3.1.1 Queries

Instance methods can be divided into two kinds: *queries*, which evaluate an object and return a value derived from its state, and *modifications*, which update the state of an object. As queries cause no state change, they have the same structure regardless of whether the data is mutable or immutable.

For instance, suppose we want to find the size of a linked list. Although "size" is not a verb, we will use the name `size` for this method to match the collections in the Java standard libraries. As with all things, the initial shape of the function should match the shape of the data: the `size()` method of a `LinkedList` should do something with its `head` to find its answer. If the `head` is `null`, then the list is empty and its size is zero. Otherwise, query that node for the size of the portion of the list that begins with that node. (In this case, that is the entire list!) This means that a `Node` should also have a `size` method, which again should match the shape of the data. The value stored in this node (the `data` field) is irrelevant. If the `rest` of the list is empty (`null`) then the size is one, from the node being queried, otherwise it is one plus the size of the `rest` of the list.

With this in mind, we write two very similar methods, one for the wrapping `LinkedList` and one for the inner `Node`.

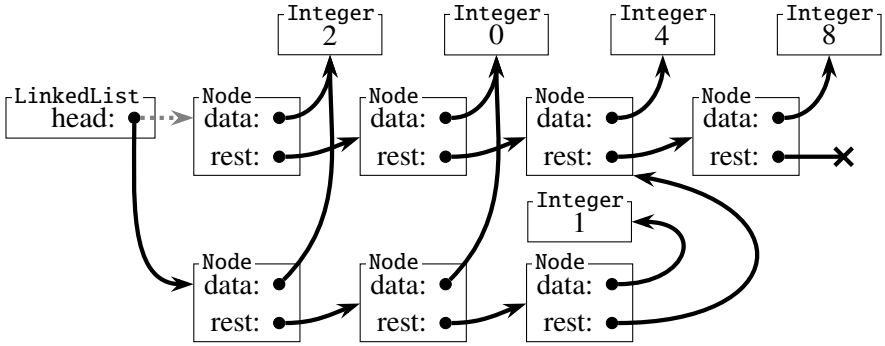

Figure 3.2: Adding a new node into a linked list, with persistence.

⟨*methods of the LinkedList class* 35a⟩≡ 36a▸ 35a

```
public int size() {
    if (head == null) { return 0; }
    return head.size();
}
```

⟨*methods of the Node class* 35b⟩≡ 36b▸ 35b

```
private int size() {
    if (rest == null) { return 1; }
    return 1 + rest.size();
}
```

3.1.2 Modifications

In a persistent data structure, objects are never changed. Instead, new objects are returned, which might share some structure with the original object. If we are using persistence on internal nodes but exposing a mutable interface, this means that the `head` of a `LinkedList` can change, but no aspect of any `Node` can ever be modified. How, then, can we add a new element, 1, between the 0 and the 4 in Figure 3.1?

Figure 3.2 depicts one solution: A new node containing 1 is created that points to the node containing 4. The node containing 0 cannot be updated to point at the new node, so another new node, containing that same 0, is created to point at the new 1-node. Similarly, the node containing 2 cannot be updated to point at the new 0-node, so a new 2-node is created. The `head` field of the list can be updated, so it is made to point to this new 2-node; the gray dotted arrow depicts the original, overwritten connection. In this way we expose a mutable API over an immutable core. The nodes no longer referenced are discarded and the garbage collector will reclaim the memory that they inhabited.

Like the size query, addition can be separated into two components. There is a method of the `LinkedList` that works with its `head`, and there is a method of the

`Node` that works with its `data` and/or the `rest` of the list. To insert a new node after an existing node, you create two nodes: one that contains the new data and points to the rest of the list, and one that contains the original data and points to the first new node. To insert a new node later on, you still create a new node: it retains the original data, but points to the new node that will be created in the next location down the line.

36a ⟨*methods of the LinkedList class* 35a⟩+≡ ◁35a

```
public void add(int index, Integer x) {
    if (index == 0) {
        head = new Node(x, head);
        return;
    }
    if (head == null) {
        throw new IndexOutOfBoundsException();
    }
    head = head.addAfter(index - 1, x);
}
```

36b ⟨*methods of the Node class* 35b⟩+≡ ◁35b

```
private Node addAfter(int index, Integer x) {
    if (index == 0) {
        return new Node(data, new Node(x, rest));
    }
    if (rest == null) {
        throw new IndexOutOfBoundsException();
    }
    return new Node(data, rest.addAfter(index - 1, x));
}
```

The instance method of a node makes no modifications. It only makes new objects. Assuming a successful addition, it always returns a newly created object, never a node from the original structure, although the data and some nodes from the end of the list may be shared. To insert a node at index n, the first n nodes are duplicated, then the newly inserted node is created, and any nodes remaining are shared between the original structure and the new structure. Eventually, after all of the cloned nodes are created, the `head` of the `LinkedList` is modified to point to the new beginning.

One advantage of the functional style is that, because objects do not change, it is easy to implement an undo system. If instead of discarding the old `head` value, we were to retain it somewhere, then we could undo the addition by simply restoring the `head`. Another advantage is that multiple objects can share some internal state without worrying about whether that state will change out from under them. This *copy-on-write* technique is used to great effect in modern computer filesystems and memory management systems.

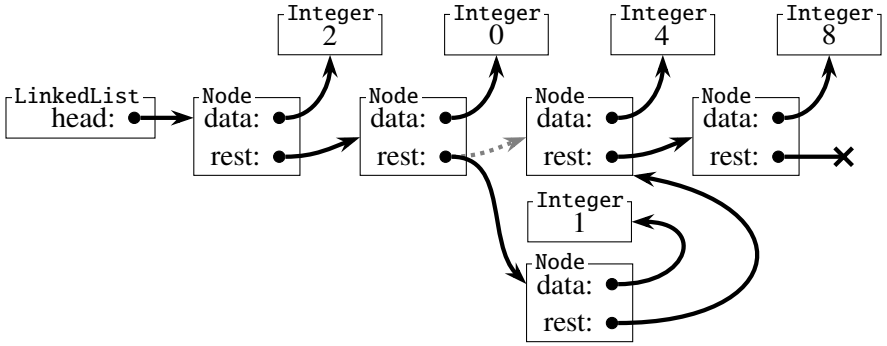

Figure 3.3: Adding a new node into a linked list, with ephemerality.

3.2 Modification of Ephemeral Structures

It is always permissible to work entirely in the functional style. However, the use of ephemeral structures can reduce the number of objects that must be allocated and potentially improve performance. Figure 3.3 depicts the insertion of a node into a linked list with ephemeral nodes. Only one new node is created, holding the new value, and its predecessor is redirected to point to this new node. Just like with persistent nodes, if the newly inserted node were placed at index zero, then it becomes the head of the list.

⟨*methods of the ephemeral LinkedList class* 37a⟩≡ 37a

```
public void add(int index, Integer x) {
    if (index == 0) {
        head = new Node(x, head);
        return;
    }
    if (head == null) {
        throw new IndexOutOfBoundsException();
    }
    head.addAfter(index - 1, x);
}
```

⟨*methods of the ephemeral Node class* 37b⟩≡ 37b

```
private void addAfter(int index, Integer x) {
    if (index == 0) {
        rest = new Node(x, rest);
        return;
    }
    if (rest == null) {
        throw new IndexOutOfBoundsException();
    }
    rest.addAfter(index - 1, x);
}
```

Notice that the `LinkedList` method differs only slightly from the version using persistent nodes: with ephemeral nodes, the `head` is only updated if the new node comes first, while with persistent nodes, it is always updated. In constrast, the `Node` method is very different between the two. They have the same shape, but the method for ephemeral nodes looks suspiciously similar to the `LinkedList` method, where `head` is swapped for `rest`. Instead of returning a new node, it walks to the desired position and inserts a new node by updating the `rest` reference.

3.3 Making Copies

As we saw when first discussing box-and-arrow diagrams, and when adding to a persistent linked list, assigning an object of reference type to a variable does not produce a copy of that object, only a new reference to it. This is why, in Figures 1 and 3.2, there are multiple arrows pointing to the same boxes. However, sometimes we do want an actual copy of a value. The semantics of copying are generally understood to be that the copy is equivalent to the original but independent from it. That is, they are representations of the same value, but modifying one should not propagate as a modification of the other. We will discuss equivalence, and the associated `equals(...)` method later in another chapter; for now we focus on independence, using a constructor that takes another list.

Persistent data structures make having independent copies simple. The copy-on-write approach to modification means that a field-by-field copy suffices. This is often called a *shallow copy*.

38a ⟨*constructors of the persistent LinkedList class* 38a⟩≡
```
public LinkedList(LinkedList other) { head = other.head; }
```

If the list created by this constructor is modified, then new nodes will be constructed and its head will be directed to one of the new nodes. The original structure remains unmodified, so the other list sees no change. Similarly, if the other list is modified, then this list sees no change.

In contrast, trying to use that approach for lists of ephemeral nodes could be a problem. If the new list and the old list have the same node as their head, then adding a new node anywhere after that head node will result in a change seen by both lists. Instead, a *deep copy* must be performed, creating a clone of every existing node.

38b ⟨*constructors of the ephemeral LinkedList class* 38b⟩≡
```
public LinkedList(LinkedList other) {
    if (other.head == null) {
        head = null;
        return;
    }
    head = new Node(other.head);
}
```

⟨*constructors of the ephemeral Node class* 39⟩≡ 39

```
public Node(Node other) {
    if (other == null) {
        throw new NullPointerException();
    }
    data = other.data;
    if (other.rest == null) {
        rest = null;
    } else {
        rest = new Node(other.rest);
    }
}
```

Whether the lists are written in persistent or ephemeral style, if the actual contained values were mutable then we would not have achieved full independence using these constructors, as each copy of the list refers to the same instances of the values. However, `Integer` values are immutable, so this is not an issue here. For our lists of integers, this approach suffices to achieve equivalence and independence.

3.4 Variations

All methods written in this chapter to this point have assumed that the marker for the end of the list is having a null reference for the `rest` of the list. However, this causes the end of the list to be a special boundary condition, where it might not be safe to call a method on the `rest`. To avoid this literal edge case, a trailing *sentinel* node can be used. This is a specially marked non-data node whose presence signals the end of a list. A leading sentinel node can be used as well to avoid having the `head` be an edge case. This would avoid the code duplication seen in the case of insertion into an ephemeral list.

Another way to avoid the possibility of a null reference is to have a circular list, where the last node in the chain points back to the first like a necklace. Like having both leading and trailing sentinels, this guarantees that any insertion occurs strictly between two existing nodes (which may in this case mean "between a node and itself"). Often a single sentinel is still used with circular lists to avoid ever having a null reference, even in the case of the empty list.

The lists that we have seen so far have been *singly linked lists*. A *doubly linked list* augments this structure to have nodes point back to the previous element as well. The memory cost to store all the reverse links is high, but in the rare occasion that the doubly-linked list is the best data structure for the job, it is nice to have around!

3.5 Some List Operations

This chapter has provided concrete implementations for querying the size of a list and for inserting elements into a list. These operations and some other useful instance methods are as follows. All operations that take an index should throw `IndexOutOfBoundsException` if the index is negative or too large.

Queries:

- `Integer get(int index)`
  Returns the element at the specified location.
- `boolean isEmpty()`
  Returns whether the list contains any nodes.
- `int size()`
  Returns the number of nodes in the list.

Modifications:

- `boolean add(Integer x)`
  Appends `x` to the end of the list and returns `true`.
- `void add(int index, Integer x)`
  Inserts `x` at the specified location in the list.
- `void clear()`
  Removes all elements from the list.
- `Integer remove(int index)`
  Removes and returns the value at the specified location in the list.
- `Integer set(int index, Integer x)`
  Overwrites the value at the given location with `x` and returns the original value.

Consider how to implement these modifications on a persistent node structure and on an ephemeral node structure. Some of these methods can be implemented in terms of others; those methods would not visibly differ between the two approaches, although their inner workings might differ considerably.